Binary Search

Search Insert Position	—> Easy
Search a 2D Matrix	—> Medium
Find Peak Element	—> Medium
Search in Rotated Sorted Array	—> Medium
Find First and Last Position of Element in Sorted Array	—> Medium
Find Minimum in Rotated Sorted Array	—> Medium
Median of Two Sorted Arrays	—> Hard

35. Search Insert Position.

eg

$\{1, 3, 5, 6\}$, target = 2

0 1 2 3

1 < 2 True

3 < 2 False → So insert target at this index

O/P = 1 → index

eg

$\{1, 3, 5, 6\}$, target = 1

0 1 2 3

1 == 1 return that index

O/P = 0

eg

$\{1, 3, 5, 6\}$, target = 4

0 1 2 3

1 < 4 ✓

3 < 4 ✓

5 < 4 ✗ → insert

O/P = 2

DATE: __/__/__
PAGE: _____

Code

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
      int low=0 ;     int high = nums.size()-1;
      while(low <= high) {
          int mid = (low + high) /2;
          if (target == nums[mid]) return mid;

          if (nums[mid] < target) {
                    low= mid+1; → Target in right half
            }
          else { high = mid-1; → Target in left half
              }
      }
      return low ;
    }
};
```

DATE: / /
PAGE:

74. Search a 2D Matrix

i/p:-

	0	1	2	3
0	1	3	5	7
1	10	11	16	20
2	23	30	34	60

, target = 11 ⇒ (sorted)

o/p: True

⊙ integers are sorted in every row

⊙

1	3	5	7
10	11	16	20
23	30	34	60

= eg element 10 का previous row i.e R1 then R1 का last element i.e 7 10 < 7 (10 छोटा होगा 7 से)

→ sorted
→ sorted
→ sorted

open ⇒ sorted: 1 3 5 7 10 11 16 20 23 30 34 60

(1st way)

target = 11

				3
0	1	3	5	7
	10	11	16	20
	23	30	34	60

i=0, j=3
11 > 7
So move to next row as it is sorted
So we find 11 in next row

⇒

				3
	1	3	5	7
1	10	11	16	20
	23	30	34	60

⊙ i=1, j=3
11 > 20 False
So, j--

⊙ i=1, j=2
11 > 16 False
So j--

⊙ i=1, j=1
11 ≥ 11 (True)
return True

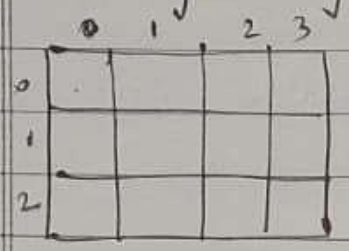
Start i=0, j=n-1

```
if(mat[i][j] > target) {
        j--;
}
elsef(mat[i][j] < target) {
        i++;
}
else {
        return True;
}
return false.
```

⇒ T.C = $O(n^2)$ → worst case

DATE
PAGE

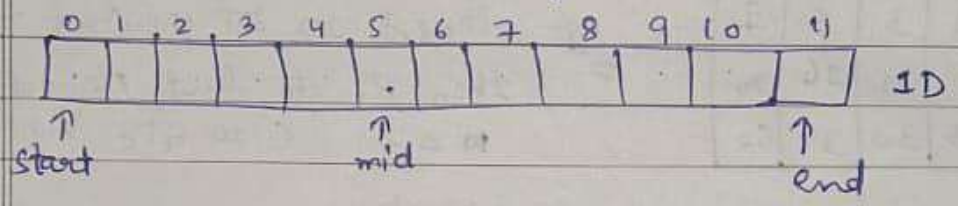
2nd way → By Binary Search.



$m \times n$
$m = 3$
$n = 4$

2D

Consider it as 1D array



start = 0 ;

end = m * n - 1 = 3 * 4 - 1 = 11

mid = start + $\frac{(\text{end} - \text{start})}{2}$ = 0 + $\frac{(11-0)}{2}$ = 5

~~2D → 1D~~ 1D का array ⇒ 2D के index में convert किया।

↳ mid/n ↙ where n = no. of column

mid % n

So, mid/n = 5/4 = 1 , mid % n = 5/4 = 1

[1] [1]

apply binary search.

```
while (start <= end) {
 mid =
 if(matrix[mid/n][mid%n] > target) {
        end = mid - 1;          → go to left side
 }
 else if (matrix[mid/n][mid%n] < target) {
        start = mid + 1;        → go to right side
 }
 else {
        return True;
 }
return false;
```

DATE: __/__/__
PAGE: _____

1st way ⇒ O(m*n)

```
{ public:
    bool searchMatrix(vector<vector<int>> & matrix, int target){
      int m = matrix.size();
      int n = matrix[0].size();
      int i=0 , j=n-1;
      while ( i<m && j>=0){
          if (matrix[i][j] > target) { j--; }
          else if (matrix[i][j] < target) { i++; }
          else { return true; }
      }
      return false;
    }
};
```

2nd way ⇒ using binary search → O(log (m*n))

```
bool searchMatrix(vector<vector<int>> & matrix, int target){
  int m = matrix.size();        int n = matrix[0].size();
  int start = 0;        int end = m*n-1;
  while ( start<=end ) {
      int mid = start + (end-start)/2;
      int row = mid/n;        int col = mid % n;
     if(matrix[row][col] > target) {
                end= mid-1;
        }
     else if (matrix[row][col] < target) {
                start = mid+1;
        }
     else {
                return true;
        }
  }
  return false;
}};
```

DATE:
PAGE:

162. Find the peak element
Peak element is one which is greater than both, left and right neighbours of itself.

Corner cases:-
- Ascending array: last element is peak
- Descending array: last element is peak
- For all same elements: each element is peak.

eg arr[] = [10, 20, 15, 2, 23, 90, 67]
↑ Peak ↑ Peak

Strategy
use binary search, analyze indices from 0 to n-1
Compute mid and for each mid
- check if arr[mid] is a peak element
- else if arr[mid-1] > arr[mid],
  ↳ check left for peak
- else if arr[mid] < arr[mid+1],
  ↳ check right for peak

eg arr = [1, 3, 20, 4, 1, 0]
0 1 2 3 4 5
↑ start ↑ mid ↑ end

start = 0, end = 5
mid = 2

20 > 3 → True (left)
20 > 4 → True (right)

o/p = 20

DATE: __/__/__
PAGE: _____

(1) checks [if nums[mid]] is a peak element

(a) [mid == 0 (or) || nums[mid-1] <= nums[mid]] left condition

either it is the first element or it is greater than or equal to previous element

eg nums = [5, 2, 1]

mid = 0 + (2 - 0)/2 = 1

nums[mid] = 2

2 > 5 False → left 2 > 1 True

5 <= 2 (False) → nums[mid-1] <= nums[mid]

(b) [mid == n-1 || nums[mid+1] <= nums[mid]] (right condition)

either it is the last element or it is greater than or equal to next element

If both conditions are true, nums[mid] is a peak → return its index.

Code T.C = O(log(n)), S.C = O(log n) → due to recursion

```
public:
  int findPeak( vector<int> &nums , int low, int high, int n){
   int mid = low + (high - low) /2;
   if((mid ==0 || nums[mid -1] <= nums[mid]) &&
     (mid == n-1 || nums[mid +1] <= nums[mid] )) {
            return mid;
   }
   else if (mid > 0 && nums[mid-1] > nums[mid]) {
        return findPeak (nums, low, mid-1 , n);
        ↳ search(left)
   }
   else {
        return findPeak(nums , mid+1 , high , n) ;
        ↳ search(right)
   }
}
```

DATE :
PAGE :

```
int findPeakElement(vector<int>& nums){
    int n = nums.size();
    int low = 0;
    int high = n-1;
    return findPeak(nums, low, high, n);
}
};
```

Imp 33. Search in a Rotated sorted array

i/p :-

0	1	2	3	4
3	4	5	1	2

, target = 4

o/p : 1 (index)

3	4	5	1	2

this part is sorted so apply here binary search

this part is sorted so apply here Binary search

3 → divide from here minimum value of 2nd part

3	4	5	1	2

↑ Pivot

Pivot_index = 3

1st part → 0 to (Pivot_index - 1)

2nd part

Pivot_index - n-1

DATE: / /
PAGE:

for ~~Pivot~~ Pivot index

Find minimum from leetcode 153
So by finding minimum
we apply binary search for both parts

Code

```
Class Solution {
public:
    int findPivot(vector<int>& nums, int n) {
        int l=0, r=n-1;
        while(l<r){
            int mid= l+(r-l)/2;
            if (nums[mid] > nums[r]) {
                l=mid+1;
            }
            else {
                r=mid;
            }
        }
        return r;
}

int binarySearch(int l, int r, vector<int>&nums, int target){
    int idx = -1;
    while(l<=r) {
        int mid = l+(r-l)/2;
        if (nums[mid] == target) {
            idx= mid;
            break;
        }
        else if( nums [mid] < target) {
```

DATE: / /
PAGE:

```
        l = mid + 1;
    }
    else {
        r = mid - 1;
    }
}
return idx;
}

int search(vector<int>&nums, int target){
    int n = nums.size();
    int pivot_index = findPivot(nums, n);
if(nums[pivot]== target) return pivot;
    int idx = binarySearch(0, pivot_index -1,
                    nums, target);
                    ↳ left side of pivot index
    if (idx != -1)
        return idx;
    idx = binarySearch(pivot_index, n-1,
                    nums, target);
                    ↳ right side including pivot
                      index.
    return idx;
}
};
```

Find first and last position of element in sorted array

$O(\log(n))$

i/p

0	1	2	3	4	5
5	7	7	8	8	10

, target = 8

o/p

0	1
3	4

← index

Binary search

```
mid = l + (r-l)/2;
if (arr[mid] > target)
        r = mid-1;
elseif( arr [mid] < target)
        l = mid+1;
else{
        return mid;
}
```

eg

0	1	2	3	4	5
0	2	2	2	2	2
l	←	mid	→		r

, target = 2

$mid = l + (r-l)/2 = 0 + (5-0)/2 = 2$

need leftmost target and rightmost target

For leftmost index :-

mid equal to target i.e = 2

l		Mid			r
0	2	2	2	2	2

→ हो सकता हो कि यही मेरा answer हो
left में जाके देखें की 2 is present or not
if present then take that 2 for leftmost

leftmost = 2 (assume)
(index)

DATE: / /
PAGE:

go to left side

⇒

0	2	2	2	2	2

l r mid

mid = 0 + (1 - 0) / 2 = 0

mid

0	2	2	2	2	2

l r

target > mid ⇒ 2 > 0 True go to right side

l = mid + 1

mid

0	2	2	2	2	2

l r

mid = 1 + (1 - 1) / 2 = 1

~~leftmost = r → index~~

mid

0	2	2	2	2	2

l r

arr[mid] == target

2 == 2 (True)

leftmost = 1 → (index)

update

r = mid - 1 ⇒

mid

0	1	2	3	4	5
0	2		2	2	2

r l

r = 0

l = 1 out of loop

while l ≤ r)

So, we get our leftmost index = 1

Illy for rightmost index:

0	1	2 (mid)	3	4	5
0	2	2	2	2	2
l					r

mid = 2

rightmost index = 2 → let
↳ store index

l = mid + 1

⇒

0	2	2	2	2	2
		↑ mid, l			r

mid = 3 + (5-3)/2 = 4

0	1	2	3	4	5
0	2	2	2	2	2
		l	mid		r

↳ equal to target

(update) rightmost index = 4

l = mid + 1

0	2	2	2	2	2
				mid	
				r ↖ l	

⇒ mid = 5 + (5-5)/2 = 5

0	2	2	2	2	2
					mid
			l		r

→ equal to target

rightmost index = 5
↳ update

l = mid + 1

0	2	2	2	2	2
					mid
				r	l

out of bound

DATE : __/__/__
PAGE : _____

153. Find Minimum in Rotated Sorted array

0	1	2	3	4
3	4	5	1	2

if :

$O(\log n)$

$ans = 1 \rightarrow$ minimum element

if sorted $\Rightarrow$ 1, 5, 7, 8, 10 (start, end)

smaller ← 5 → larger no.

eg

0	1	2 (mid)	3	4
3	4	5	1	2
l				r

$mid = l + (r-l)/2 = 0 + (4-0)/2 = 2$

$\Rightarrow arr[mid] = 5 > arr[r]$

$(5 > 2)$ So, our minimum value lies at right side

```
if (arr[mid] > arr[r]) {
        l = mid+1 ;
}
```

3	4	5	1	2
			l	r.

Now,

3	4	5	1 (l, mid)	2 (r)

$mid = 3 + (4-3)/2 = 3$

$arr[mid] = 1$ $arr[r] = 2$

$1 < 2$ correct $\Rightarrow$ Chances ई कि mid answer हो सकता है।

```
else {
    r = mid ;
}
```

Imp ☆

3	4	5	1 (mid, l, r)	2

$\Rightarrow mid = 3 + (3-3)/2 = 3$

$arr[mid] = 3$ $arr[r] = 1$

$3 > 1$ (False)

else $r = mid$

i.e $r = 3$ again

So while $(l < r)$ ← to avoid ∞ loop

```
int l=0 , r=n-1;
  while(l<r){
  int mid= l +(r-l)/2;
     if (arr[mid]>arr[r]){
          l=mid+1;
     }
     else{
          r=mid;
     }
  }
  return arr[r];
}
```

Binary Search

4. Median of Two Sorted Arrays (Hard)

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return **the median** of the two sorted arrays.

The overall run time complexity should be `O(log (m+n))`.

Example 1:

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

Example 2:

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) /
2 = 2.5.
```

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- `0 <= m <= 1000`
- `0 <= n <= 1000`
- `1 <= m + n <= 2000`
- `106 <= nums1[i], nums2[i] <= 106`

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<
int>& nums2) {
```

```
        // Ensure nums1 is the smaller array
        if (nums2.size() < nums1.size()) {
            return findMedianSortedArrays(nums2, nums1);
        }

        int m = nums1.size();
        int n = nums2.size();
        int start = 0, end = m;

        while (start <= end) {
            int part1 = (start + end) / 2;
            int part2 = (m + n + 1) / 2 - part1;

            // Left and Right boundaries
            int maxLeftNums1 = (part1 == 0) ? INT_MIN : nums1
[part1 - 1];
            int maxLeftNums2 = (part2 == 0) ? INT_MIN : nums2
[part2 - 1];
            int minRightNums1 = (part1 == m) ? INT_MAX : nums
1[part1];
            int minRightNums2 = (part2 == n) ? INT_MAX : nums
2[part2];

            // Check if we have found the correct partition
            if (maxLeftNums1 <= minRightNums2 && maxLeftNums2
<= minRightNums1) {
                if ((m + n) % 2 == 0) {
                    return ( (double)(max(maxLeftNums1, maxLe
ftNums2) + min(minRightNums1, minRightNums2)) ) / 2.0;
                } else {
                    return (double) max(maxLeftNums1, maxLeft
Nums2);
                }
            }
            // If left partition is too large, move left
            else if (maxLeftNums1 > minRightNums2) {
```

```
                end = part1 - 1;
            }
            // If left partition is too small, move right
            else {
                start = part1 + 1;
            }
        }

        throw invalid_argument("Input arrays are not sorted o
r valid");

    }
};
```

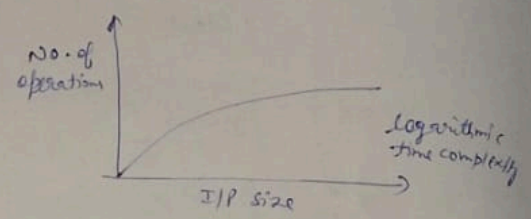
Median of two sorted arrays

Binary search
↳ works on sorted list

① Median = { 1, 2, 3, 4, 5 } For odd no.
↳ median = 3

for even ⇒ { 1, 2, 3, 4, 5, 6 }
↳ $\frac{3+4}{2} = \frac{7}{2} = 3.5$ average

② $O(\log(m+n))$



nums1 = { 1, 2, 3 } (0, 1, 2)
m = 3

num2 = { 4, 5, 9 } (0, 1, 2)
n = 3

(i) start = 0

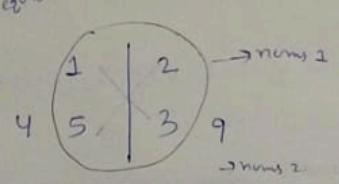
(ii) end = 3 → m

(iii) Partition 1 = $\frac{\text{start} + \text{end}}{2} = \frac{0+3}{2} = 1$ ⇒ { 1 | 2, 3

(iv) Partition 2 = $\boxed{\frac{m+n+1}{2} - \text{Partition 1}} = \frac{3+3+1}{2} - 1 = \frac{7}{2} - 1 \Rightarrow 2$

So that the elements on left side and right side are equal

⇒ { 4, 5, | 9 }



Found it

(V) maximum element from left side of an array

max leftnums 1

max leftnums 2

min rightnums 1

min rightnums 2

(vi) check

(maxleftnums 1 < (less than) minrightnums 2) && (maxleftnums 2 < minrightnums 1) }

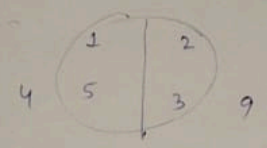
if these two conditions are true then the array is sorted

↓

then directly find median

median for → odd
→ even

here,



1 < 3 && 5 < 2 → not find median
True False

(vii) maxleftnums 1 > minrightnums 2
end = partition − 1

1 > 3
false

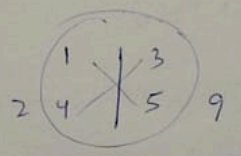
5 > 3 (true)

(viii) start = partition + 1 → 5 > 3 (true)

⇒ start = 0 + 1 = 1
end = 3

Part 1 = $\frac{1+3}{2} = \frac{4}{2} = 2$ {1 2 | 3}

Part 2 = $\frac{3+3+1}{2} - 2 = \frac{7}{2} - 2 = 1$ {4 | 5, 9}



1 < 5 && 4 < 3
False

Now check

maxleftnums 1 > minRightnums 2 1 > 5 False
end = Part − 1

(viii) start = part1 + 1 → 4 > 3 (True)

start = 1+2 = 3

end = 3

Part1 = $\frac{3+3}{2} = \frac{6}{2} = 3$ $\{1, 2, 3\}$

Part2 = $\frac{3+3+1}{2} - 3 = \frac{7}{2} - 3 = 0$ $\{4, 5, 9\}$

set as MAX

1 2 3 | 4 5 9

set → MIN

3 < 4 True

see step (vi) ⇒

find median in this case we have two middle elements so,

$\frac{3+4}{2} = \frac{7}{2} = 3.5$

⇒ kth element of two sorted arrays

kth index ka element return

int k = 5

nums1 = {2, 3, 6, 7, 9}
m = 5

nums2 = {1, 4, 8, 10}
n = 4

merged = {1, 2, 3, 4, 6, 7, 8, 9, 10}

5th smallest element = 6

m > n (5 > 4) False

left = max(0, k-n) = max(0, 5-4) = 1

right = min(k, m) = min(5, 5) = 5

left = 1, right = 5

$\{2, 3, 6 \mid 7, 9\}$ $\{1, 4, \mid 8, 10\}$

$part1 = \frac{1+5}{2} = 3$, $Part\ 2 = k - part\ 1$

$= 5 - 3 = 2$

$2\ 3\ 6 \mid 7$

$1\ 4 \mid 9, 8, 10$

$maxleftnums1 = nums1[\overset{3-1}{2}] = 6$

$maxleftnums2 = nums2[\overset{2-1}{1}] = 4$

$minrightnums1 = nums1[3] = 7$

$minrightnums2 = nums[2] = 8$